

Il test reale su dual boot

Le slide una alla volta, ognuna con la sua spiegazione

Ogni pagina ha l'immagine di una slide e sotto la spiegazione. I riquadri grigi sono **esempi con i numeri**; i riquadri gialli segnalano i **punti da correggere o da dire con cautela**; i riquadri verdi le **verifiche** fatte ricontrollando i dati.

Le parole e le unità che tornano sempre

Nodo	Il computer che riceve il pacchetto, esegue la rete neurale e lo inoltra. Nel banco è il core 6.
XDP	Il primo punto del kernel Linux in cui un programma può vedere un pacchetto appena arrivato.
Pipeline	Una delle versioni del programma che esegue la rete neurale (P1, P1.5, P2, P3).
baseline / rxonly	I due programmi di riferimento: la baseline inoltra senza rete neurale, rxonly riceve e basta.
Coda	I 256 posti davanti al nodo dove i pacchetti aspettano. Se è piena, chi arriva è respinto.
Coda d'uscita	La coda del cavo virtuale verso il nodo successivo. Con più core sul nodo ce n'è una per core, svuotata da un core suo (slide 12).
Capacità	Quanti pacchetti al secondo il nodo riesce a elaborare al massimo.
Collo di bottiglia	Il pezzo più lento della catena: decide quanto traffico passa.
M/s	Milioni di pacchetti al secondo.
ns, μ s, ms	Nanosecondi (miliardesimi di secondo), microsecondi ($1 \mu\text{s} = 1\,000 \text{ ns}$), millisecondi ($1 \text{ ms} = 1\,000 \mu\text{s}$).
Gbit/s	Miliardi di bit al secondo. Con pacchetti da 64 byte, $1 \text{ Gbit/s} \approx 1,95$ milioni di pacchetti al secondo.
Costo per pacchetto	1 diviso la capacità. 2 milioni di pacchetti al secondo = 500 ns per pacchetto.
Tempo di CPU per pacchetto	Quanto tempo il core del nodo lavora davvero per ogni pacchetto: occupazione del core $\times$ 1 diviso la capacità. Coincide con il costo per pacchetto quando il nodo lavora il 100% del tempo.
Scrittore	Chi mette pacchetti in una coda. Con una scheda di rete vera è la scheda stessa: uno per coda.

TESI · EBPF / XDP

Il test reale su dual boot e la domanda sul bit rate

Come si misura quanti pacchetti regge un nodo, con quale generatore di traffico, e che cosa succede quando il traffico offerto cresce.

Ettore Cantile

ottobre 2026

La presentazione racconta il test del datapath su traffico vero, rifatto sul portatile con Ubuntu installato direttamente (dual boot) invece che dentro una macchina virtuale. Al centro c'è il **nodo**: il computer che riceve un pacchetto, esegue la rete neurale dentro il kernel (con XDP, cioè nel primo punto in cui Linux vede il pacchetto) e decide da quale porta farlo uscire.

Le domande a cui risponde sono tre:

1. Quanti pacchetti al secondo regge un nodo, con ciascuna delle versioni del programma?
2. Con quale traffico lo misuriamo? Il generatore è xdp_gen, che consegna i pacchetti come li consegnerebbe una scheda di rete vera.
3. Che cosa succede quando il traffico cresce oltre quello che il nodo regge: dove si perdono i pacchetti? È la domanda centrale, e la risposta arriva nella parte 5.

Tutte le cifre vengono dallo stesso portatile (Intel Core Ultra 7 155H), con le condizioni della macchina controllate e registrate a ogni misura. Le misure con xdp_gen sono di una stessa sessione, il 2 ottobre; il confronto con la macchina virtuale (slide 8) usa misure precedenti, fatte con pktgen.

La domanda

LA DOMANDA

Quando il traffico cresce, dove si perde?



Dove si perde dice qual è il **collo di bottiglia**.

Si perde prima del programma

Il pacchetto non è mai entrato: il collo di bottiglia è il **programma**.

Si perde dopo il programma

Il programma lo ha elaborato, ma non è arrivato: il collo di bottiglia è la **trasmissione**.

Per rispondere si contano i pacchetti che il programma **riceve** e quelli che **rilancia**.

La risposta arriva nella parte 5; le parti 1-4 spiegano come si misura

2

Se si manda al nodo sempre più traffico, a un certo punto il nodo non ce la fa più. Davanti al nodo c'è una **coda** di 256 posti: i pacchetti che il nodo non ha ancora elaborato aspettano lì, in ordine di arrivo. Finché il nodo è più veloce del traffico la coda resta quasi vuota; quando è più lento la coda si riempie, e i pacchetti che arrivano a coda piena vengono **respinti**, cioè persi.

La slide segue questa catena in quattro passi: il traffico cresce, la coda si riempie, il ritardo cresce (ogni pacchetto aspetta quelli davanti), e alla fine si perde. La domanda è **dove** si perde, perché il punto della perdita dice chi è il collo di bottiglia:

- se si perde **prima** del programma (il pacchetto non è mai entrato), il programma è troppo lento per il traffico;
- se si perde **dopo** (il programma lo ha elaborato e deciso di inoltrarlo, ma non arriva a destinazione), il problema è la trasmissione in uscita.

Esempio. Il nodo riesce a elaborare 3,4 milioni di pacchetti al secondo e il generatore ne manda 5,6 milioni. Ogni secondo 2,2 milioni di pacchetti non trovano posto e vengono respinti. Se contiamo quanti pacchetti entrano nel programma (3,4 milioni) e quanti ne escono (sempre 3,4 milioni), vediamo che si perdono tutti prima: il collo di bottiglia è il programma.

Per rispondere servono quindi due conteggi in più rispetto a quelli del generatore: i pacchetti che il programma riceve e quelli che rilancia. Come si ottengono è nelle slide 13 e 14.

Come è organizzata la presentazione

COME È FATTA LA PRESENTAZIONE

Cinque parti, dal banco alla risposta

Ogni parte risponde a una domanda: le prime quattro preparano la risposta alla quinta.

1

Il banco

Che cosa si misura, con quali programmi e come.

Che cosa **misuriamo**?

2

La macchina

Dalla macchina virtuale al dual boot.

Perché fidarsi dei **numeri**?

3

Il generatore

xdp_gen: pacchetti come da una scheda di rete.

Con che **traffico** si misura?

4

I risultati

Quanto regge il nodo, su uno e due core.

Quanti **pacchetti** al secondo?

5

La domanda

Che cosa succede quando il traffico cresce.

Dove si **perde**?

IN BREVE

La risposta, in anticipo: si perde **prima** del programma, quindi il collo di bottiglia è **il programma**. E le misure ora si ripetono.

Le parti 2-5 raccontano ciò che è cambiato dal test in macchina virtuale

3

È la mappa delle cinque parti. Ognuna risponde a una domanda, e le prime quattro servono a potersi fidare dei numeri della quinta.

1. **Il banco** (slide 4-5): quali programmi si confrontano e come si misura il costo per pacchetto.
2. **La macchina** (slide 6-8): perché nella macchina virtuale le misure non erano affidabili, che cosa è cambiato con il dual boot e quanto sono ripetibili ora.
3. **Il generatore** (slide 9-10): xdp_gen, che consegna pacchetti come una scheda di rete, e l'alternativa, pktgen.
4. **I risultati** (slide 11-12): quanti pacchetti al secondo regge ogni programma, su un core e su due.
5. **La domanda** (slide 13-19): dove si perde quando il traffico cresce e che cosa succede al ritardo.

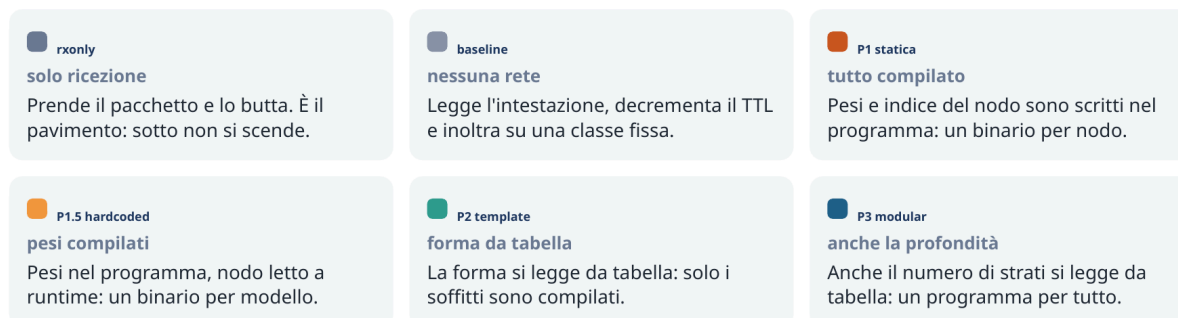
La risposta è anticipata nel riquadro in basso: si perde **prima** del programma, quindi il collo di bottiglia è il programma eBPF, non la trasmissione. E, grazie al dual boot, le misure ora si ripetono.

I sei programmi

1 - IL BANCO - I PROGRAMMI

Sei programmi, che cambiano per una cosa sola

Calcolano la stessa rete e prendono la stessa decisione. Cambia quanto sanno già **quando vengono compilati**.



Più informazioni si spostano dalla compilazione all'esecuzione, più il programma è **riutilizzabile**, e più cresce il **costo per pacchetto**.

Cinque pipeline più rxonly, che misura la sola ricezione - i colori dei quadratini tornano in tutti i grafici

4

Le quattro pipeline (P1, P1.5, P2, P3) calcolano la stessa rete neurale e prendono la stessa decisione sullo stesso pacchetto: lo verificano i test di correttezza. Cambia solo **quanto è scritto dentro il programma** quando lo si compila, e quanto invece il programma legge da tabelle (le mappe eBPF) mentre gira. Due programmi di riferimento completano il confronto.

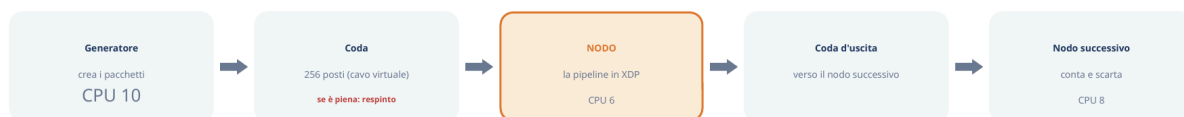
- **rxonly**: riceve il pacchetto, incrementa un contatore e lo butta. Non fa altro. Serve a sapere quanto costa la sola ricezione: nessun programma può essere più veloce, quindi è il "pavimento".
- **baseline**: legge l'intestazione, abbassa il TTL e inoltra il pacchetto su una porta fissa, senza rete neurale. Fa tutto quello che fa una pipeline tranne l'inferenza: per questo **quanto una pipeline costa in più della baseline è il costo della rete neurale**.
- **P1 statica**: pesi della rete e numero del nodo sono costanti nel codice. Il compilatore può semplificare molto (moltiplicazioni per costanti, rami eliminati), ma serve un binario per ogni nodo: quello compilato per il nodo 7 non va bene per il nodo 8.
- **P1.5 hardcoded**: i pesi sono nel codice, il numero del nodo si legge da una tabella. Un binario per modello, valido per tutti i nodi che usano quel modello.
- **P2 template**: anche la forma della rete (quanti ingressi, quanti neuroni per strato) si legge da tabella. Nel codice ci sono solo i soffitti, cioè i valori massimi ammessi, e i cicli girano fino a quelli. Un solo programma serve molti modelli.
- **P3 modular**: anche il numero di strati si legge da tabella, e ogni strato è un programma a parte raggiunto con un salto (tail call). È la versione più generale.

Regola generale: più informazioni si spostano dalla compilazione all'esecuzione, più il programma è riutilizzabile, e più lavoro fa per ogni pacchetto (letture da tabella, cicli più lunghi, salti). Le slide successive misurano proprio quanto costa questa generalità. I colori dei quadratini restano gli stessi in tutti i grafici.

Il banco di misura

1 - IL BANCO - COME SI MISURA

Il banco misura quanti pacchetti regge il nodo



1

Coda sempre piena

Il generatore offre **più pacchetti** di quanti il nodo riesca a elaborare.

2

Nodo sempre occupato

Il suo core non resta mai senza lavoro: il ritmo a cui elabora è la sua **capacità**.

3

Costo = 1 / capacità

Se elabora 5 milioni di pacchetti al secondo, spende **200 ns** per ciascuno.

QUINDI

Si misurano 300 ms a regime: costo per pacchetto = tempo ÷ pacchetti elaborati. Il generatore sta su altri core, quindi **non entra nella misura**.

Un solo portatile: generatore, nodo e nodo successivo ciascuno sul proprio core fisico

5

Tutto gira sullo stesso portatile, ma ogni pezzo ha il suo core fisico dedicato, così nessuno ruba tempo agli altri:

- il **generatore** (CPU 10) crea i pacchetti: è xdp_gen, un thread solo;
- la **coda** da 256 posti è quella del cavo virtuale (veth) che collega generatore e nodo;
- il **nodo** (CPU 6) esegue la pipeline in XDP: è l'unica cosa che vogliamo misurare;
- la **coda d'uscita** porta i pacchetti inoltrati verso il nodo successivo;
- il **nodo successivo** (CPU 8) conta i pacchetti che arrivano e li butta.

Come si misura il costo. Si manda più traffico di quanto il nodo possa elaborare. Così valgono le tre condizioni della slide: la coda è sempre piena, il core del nodo non resta mai senza lavoro, e il ritmo a cui elabora è la sua **capacità**. Il tempo speso per ogni pacchetto è allora 1 diviso la capacità.

Esempio. In una finestra di 300 millisecondi il nodo elabora 1 023 000 pacchetti con P2. Sono 3,41 milioni al secondo, quindi spende 1 / 3 410 000 di secondo per pacchetto, cioè 293 ns.

Perché mandare più traffico del necessario? Se il generatore mandasse meno pacchetti di quanti il nodo ne regge, il nodo starebbe fermo ogni tanto e misureremmo il ritmo del generatore, non quello del nodo. La misura avviene solo a regime: si scartano i primi istanti, si leggono i contatori a inizio e fine finestra, e si ripete più volte (giri) per vedere quanto varia.

Verificato. Il banco misura anche quanto è occupato il core del nodo, nella stessa finestra in cui conta i pacchetti (il **tempo di CPU per pacchetto**). Con xdp_gen il nodo lavora il 100% del tempo, e 1 diviso la capacità coincide con il tempo di CPU.

Il problema della macchina virtuale

2 · MACCHINA VIRTUALE E DUAL BOOT · IL PROBLEMA

Nella macchina virtuale il nodo ogni tanto si fermava

Quanto può stare fermo il nodo

prima che la coda da 256 posti trabocchi

Riempire la coda (1,5 M pacchetti/s)

Risveglio da sonno profondo

Pausa di un core virtuale

la coda trabocca

170 μ s310 μ s

alcuni millisecondi

Due strade, entrambe cattive

Generatore e nodo su core separati

Il nodo perdeva **fra l'8 e il 70%** dei pacchetti a qualunque ritmo, anche lentissimo.

Generatore e nodo sullo stesso core

Non si perdeva niente, ma il core faceva il lavoro di tutti e due: il collo di bottiglia era sempre **il generatore**.

QUINDI

La perdita misurava **le assenze del core**, non il programma: dentro la macchina virtuale un core può fermarsi per millisecondi, e non si vede né si può impedire.

6

Fino al 26 settembre il test girava dentro VirtualBox. Un "core" della macchina virtuale non è un core vero: è un thread del sistema che ospita la macchina virtuale, e quel sistema può fermarlo quando vuole, anche per qualche millisecondo. Dall'interno non lo si vede e non lo si può impedire.

Il grafico a sinistra mette a confronto i tempi in gioco. Quanto può stare fermo il nodo prima di perdere pacchetti? Dipende da quanto ci mette la coda a riempirsi. A 1,5 milioni di pacchetti al secondo ne arriva uno ogni 0,67 μ s, quindi 256 posti si riempiono in $256 \times 0,67 \approx 170 \mu$ s. Qualunque pausa più lunga fa traboccare la coda. Un core che si risveglia dal sonno profondo ci mette 310 μ s; una pausa del core virtuale può durare millisecondi: entrambe superano di molto il margine.

Esempio. Il core virtuale viene fermato per 2 millisecondi. In quel tempo arrivano $2 \text{ ms} \times 1,5 \text{ milioni/s} = 3\,000$ pacchetti, ma la coda ne contiene 256: circa 2\,750 vanno persi. E il programma non c'entra niente: poteva essere velocissimo.

Si poteva scegliere solo fra due strade, entrambe sbagliate:

- **generatore e nodo su core separati:** perdite dall'8 al 70% anche a traffico bassissimo, cioè si misuravano le pause del core e non il programma;
- **tutto sullo stesso core:** niente perdite, perché il core non si ferma mentre lavora per sé, ma il core faceva il lavoro del generatore e del nodo insieme, e il limite era sempre il generatore.

La conclusione è che nella macchina virtuale la perdita misurava le assenze del core. Per rispondere alla domanda serviva una macchina vera.

La soluzione: il dual boot

2 · MACCHINA VIRTUALE E DUAL BOOT · LA SOLUZIONE

Sul dual boot ogni core ha un ruolo fisso

La macchina: un processore ibrido

Intel Core Ultra 7 155H, 22 CPU: ognuna ha il suo ruolo



1 Frequenza fissa e misurata

3,5 GHz richiesti, 3 494 MHz misurati dai contatori del processore durante ogni misura.

2 Niente sonno profondo

Un core che dorme impiega 310 μ s a svegliarsi, più dei 170 μ s che la coda concede.

3 Il resto del sistema altrove

Desktop, servizi, interruzioni e thread del kernel stanno sulle altre CPU.

4 Un monitor sempre acceso

Temperatura, throttling e frequenza vera: una finestra fuori condizione si conta, non si scarta.

QUINDI

Il core del nodo **non dorme, non cambia frequenza e nessuno lo interrompe**: si misura il programma, non la macchina.

Dal 27 settembre Ubuntu gira direttamente sul portatile. Lo script `host_conditions.py` prepara la macchina prima di ogni misura e la rimette com'era alla fine (anche se la misura si interrompe):

- **ogni ruolo ha un core fisico suo** (schema a sinistra): generatore sulla CPU 10, nodo sulla 6, nodo successivo sulla 8; le altre CPU al sistema. Il processore è ibrido: 6 P-core veloci, ciascuno con due thread gemelli (SMT), più 8 E-core e 2 LP E-core più lenti. Il gemello di ogni core usato resta a riposo (in tratteggio), perché due thread sullo stesso core fisico si dividono pipeline e cache e si rallentano a vicenda. Con due core sul nodo (slide 12) si aggiungono la CPU 8 al nodo, la 1 al generatore e le CPU 3 e 5 all'uscita;
- **frequenza fissa**: 3,5 GHz richiesti, e si verifica con i contatori del processore (APERF/MPERF) che sia davvero quella: 3 494 MHz misurati durante le misure. Senza frequenza fissa il turbo cambierebbe la velocità del core da un istante all'altro;
- **niente sonno profondo**: gli stati di risparmio energetico profondi (C6, C10) sono spenti sui core del banco, perché un core che si addormenta ci mette fino a 310 μ s a svegliarsi, più dei 170 μ s che la coda concede (slide 6);
- **tutto il resto altrove**: processi, thread del kernel, interruzioni e code di lavoro del sistema vengono spostati sulle CPU di sistema;
- **un monitor sempre acceso**: ogni secondo registra temperatura, throttling termico e frequenza vera. Se una finestra di misura avviene in condizioni sbagliate non si scarta di nascosto: viene contata e segnalata nel file `env.csv` del run.

Risultato: il core del nodo non dorme, non cambia velocità e nessuno lo interrompe. Si misura il programma, non la macchina.

Prima e dopo

2 · MACCHINA VIRTUALE E DUAL BOOT · PRIMA E DOPO

Con il dual boot le misure si ripetono

Macchina virtuale			Dual boot		
Pipeline	costo	variazione	costo	variazione	
P1 statica	~82 ns	160%	49 ns	2,2%	
P1.5	~116 ns	104%	58 ns	4,3%	
P2	~430 ns	19%	227 ns	2,9%	
P3	~670 ns	15%	350 ns	3,6%	

Costo sopra la baseline, per pacchetto. Le cifre assolute non si confrontano (frequenza e metodo erano diversi): si confronta la variazione.

160% → 2%

variazione del costo di P1 fra le esecuzioni

1-2% → 0

pacchetti respinti sotto capacità, anche per il solo contatore

58% · 0,2% · 7,7%

perdita dello stesso punto in tre giri (macchina virtuale). Sul dual boot: in genere entro lo 0,1%, al massimo 6%

QUINDI

Il guadagno vero non è la velocità, è la **ripetibilità**: differenze che prima sparivano nel rumore, come P1 contro P1.5 o scartare contro inoltrare, ora si misurano.

Macchina virtuale: 18 settembre, pktgen, 512 byte · dual boot: 2 ottobre, xdp_gen, 64 byte · variazione fra le esecuzioni

8

Da quale test vengono i numeri. "Costo" è quanti nanosecondi per pacchetto una pipeline spende in più della baseline, cioè il costo della rete neurale. "Variazione" dice quanto cambia quel costo ripetendo la stessa misura. La colonna della macchina virtuale viene dalle misure di allora, fatte con pktgen a 512 byte; quella del dual boot dalle misure con xdp_gen del 2 ottobre, a 64 byte. Le cifre assolute delle due colonne non si confrontano (generatore, frequenza e taglia erano diversi): si confronta la variazione.

Esempio. Nella macchina virtuale P1 una volta costava 164 ns sopra la baseline e un'altra 32 ns: variazione del 160%. Con numeri così non si può dire se P1 sia più veloce di P1.5. Sul dual boot i tre giri danno 48, 49 e 50 ns: variazione del 2%, e la differenza di circa 9 ns fra P1 e P1.5 si vede.

Gli altri numeri a destra:

- **160% → 2%:** la variazione del costo di P1 fra le esecuzioni, prima e dopo;
- **1-2% → 0:** con un traffico più basso della capacità, nella macchina virtuale si perdeva comunque l'1-2% dei pacchetti (per le pause del core), anche con il solo contatore; sul dual boot nessuno;
- **58% · 0,2% · 7,7%:** nella macchina virtuale lo stesso punto del test sul bit rate, ripetuto tre volte, dava tre perdite completamente diverse. Sul dual boot lo stesso punto, nei 5 giri, varia in genere meno dello 0,1%, al massimo del 6% vicino al punto in cui la pipeline smette di reggere.

Il messaggio della slide: il guadagno vero del dual boot non è la velocità, è la **ripetibilità**. Differenze che prima sparivano nel rumore, come P1 contro P1.5 o scartare contro inoltrare (circa 50 ns), ora si misurano.

Il generatore: xdp_gen

3 · IL GENERATORE · XDP_GEN

xdp_gen consegna pacchetti già nel formato di XDP



15-19

milioni di pacchetti al secondo
offerti da un solo thread

293 ns

P2 con xdp_gen, contro 529 ns
con pktgen

QUINDI

Il costo del generatore non passa più dal nodo. Si usa un thread per coda: più thread nella stessa coda si intralciano e rallentano anche il nodo.

Frame grezzi creati con BPF_PROG_TEST_RUN in modalità live frames · script ipa/test/xdp_gen.py

9

xdp_gen (script `ipa/test/xdp_gen.py`) usa una funzione del kernel, `BPF_PROG_TEST_RUN` in modalità **live frames**: il kernel prepara dei pacchetti già nel formato che XDP si aspetta (un frame grezzo con 256 byte liberi davanti) e li spedisce davvero sul cavo virtuale. La slide segue i tre passi: xdp_gen crea il frame, il cavo lo passa così com'è, il nodo lo elabora. Niente skb e niente copia: il pacchetto arriva al nodo come arriverebbe da una scheda di rete vera con XDP nativo, e il nodo paga solo il proprio lavoro.

I due numeri in basso: un solo thread di xdp_gen offre **15-19 milioni** di pacchetti al secondo, più di quanto ogni pipeline regga su un core; P2 costa **293 ns** per pacchetto (con l'alternativa, pktgen, 529: slide 10).

Si usa così: **un thread per ogni coda del nodo**. La coda in cui un thread scrive dipende dalla CPU su cui gira (numero della CPU modulo numero di code), e con più thread nella stessa coda i thread si passano un lucchetto e le stesse righe di cache, e rallentano anche il nodo che la svuota.

Il ritmo. Per il test al crescere del traffico (slide 15-19) serve un ritmo fissato. xdp_gen non ne ha uno suo: lo tiene il suo programma, che a ogni giro guarda l'orologio, scarta il pacchetto se non è ancora il momento e lo spedisce quando lo è. I pacchetti partono a distanza regolare, non a raffiche; nello stesso istante il programma ci scrive un timbro con l'ora di partenza, e il nodo successivo misura il ritardo. Il ritmo è esatto fino a circa 11 milioni di pacchetti al secondo.

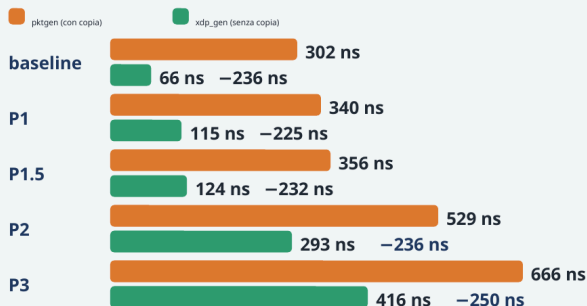
Verificato il 30 settembre e il 2 ottobre. Con tre thread nella stessa coda il nodo si fermava intorno a 7 milioni al secondo: era la coda condivisa, non il generatore. E fino al 1 ottobre il nodo risultava occupato all'87% anche a coda piena: a ogni chiamata con cui xdp_gen si fa dare i pacchetti dal kernel il generatore restava fermo circa 12 ms, e il nodo intanto svuotava la coda e dormiva. Lo hanno mostrato le statistiche di scheduling del thread del nodo (dormiva tante volte quante erano le chiamate). Ora ogni misura è una sola chiamata, e il nodo lavora al 100%.

L'alternativa: pktgen

3 · IL GENERATORE · L'ALTERNATIVA

L'alternativa, pktgen, costa 225–250 ns in più

Nanosecondi per pacchetto, stesso programma



pktgen: il generatore di Linux

Crea una skb, e il cavo virtuale la copia prima del nodo: lo stesso programma costa di più.

La rete neurale costa uguale

Sopra la baseline: xdp_gen +49, +58, +227, +350 ns; pktgen +38, +53, +226, +364.

Da dove vengono i 225–250 ns

Dalla copia della skb e dal percorso dello stack: il costo del generatore, pagato sul core del nodo.

QUINDI

Il banco usa **xdp_gen**, che non ha la copia. pktgen resta un'alternativa: sopra la baseline dà la stessa risposta.

pktgen 28/9, xdp_gen 2/10 (nodo al 100%) · P1: 340 – 115 = 225 · P1.5: 356 – 124 = 232 · P2: 529 – 293 = 236 · P3: 666 – 416 = 250 ns

10

pktgen è il generatore di traffico che sta dentro il kernel Linux. Crea i pacchetti come li crea il resto del sistema, dentro una struttura chiamata skb, che lascia 64 byte liberi davanti ai dati. XDP però ne vuole 256, quindi il cavo virtuale **copia** ogni pacchetto in una pagina nuova prima di consegnarlo al nodo, e la copia la paga il core del nodo.

Il grafico mette a confronto lo stesso programma con i due generatori, su un core. La barra arancione (pktgen) è sempre più lunga di quella verde (xdp_gen) di **225–250 ns**: è il costo della copia e del percorso della skb, uguale per tutte le pipeline.

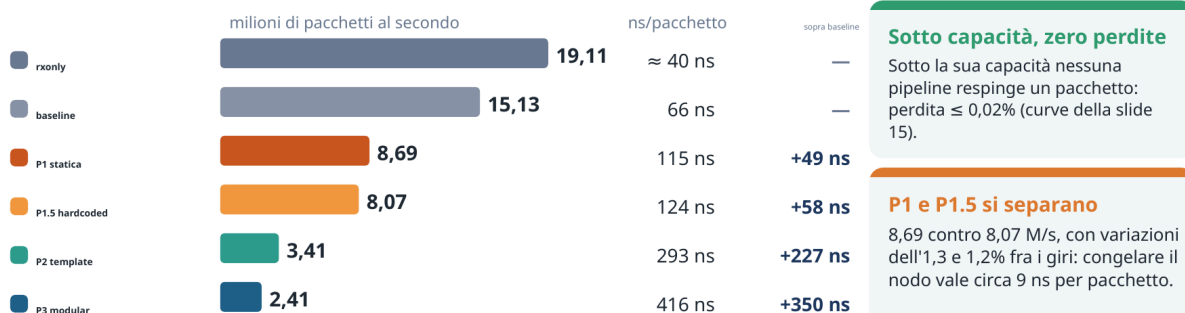
Esempio. P2: 529 ns con pktgen, 293 con xdp_gen, differenza 236. Baseline: 302 contro 66, differenza 236. P3: 666 contro 416, differenza 250 (P3 varia di più fra una sessione e l'altra).

Sopra la baseline la rete neurale costa uguale con i due generatori: +38, +53, +226 e +364 ns con pktgen, +49, +58, +227 e +350 con xdp_gen, entro la variazione fra sessioni. Per questo il banco usa xdp_gen, che dà il costo come lo pagherebbe un nodo con una scheda vera; pktgen resta un'alternativa e un controllo. Le cifre di pktgen sono del 28 settembre, quelle di xdp_gen del 2 ottobre.

Risultati con xdp_gen

4 - I RISULTATI SUL DUAL BOOT - XDP_GEN

Con xdp_gen le pipeline saturano e si ordinano



QUINDI

Sopra la baseline, la rete neurale costa ~50–60 ns con i pesi compilati (P1, P1.5), ~230 ns con la forma letta da tabella (P2), ~350 ns con anche la profondità (P3).

3 giri, 64 byte, 2 ottobre · un thread di xdp_gen, uscita su un core suo · nodo al 100% · variazione fra i giri 1,2–3,1%

11

Per ogni programma la slide riporta quanti milioni di pacchetti al secondo regge (la barra), quanti nanosecondi spende per pacchetto (1 diviso la capacità) e quanto costa in più della baseline, cioè il costo della rete neurale. Un core del nodo, un thread di xdp_gen, uscita su un core suo, pacchetti da 64 byte, 3 giri. Il nodo lavora al 100%.

Esempio di calcolo. P2 regge 3,41 milioni di pacchetti al secondo, quindi spende $1 / 3\,410\,000 \text{ s} \approx 293 \text{ ns}$ per pacchetto. La baseline ne spende 66. La differenza, $293 - 66 \approx 227 \text{ ns}$, è il tempo che P2 dedica alla rete neurale: ricevere, leggere l'intestazione e inoltrare lo fanno tutte e due allo stesso modo.

Le tre letture della slide:

- **Il costo dipende da dove stanno i pesi e la forma.** Pesi scritti nel codice (P1, P1.5): la rete costa 50–60 ns. Forma letta da tabella (P2): circa 230 ns, perché i cicli girano fino ai soffitti e ogni peso è una lettura da mappa. Anche la profondità da tabella (P3): circa 350 ns, per i salti fra uno strato e l'altro.
- **Sotto capacità, zero perdite:** finché il traffico resta sotto la capacità di una pipeline nessun pacchetto viene respinto ($\leq 0,02\%$, curve della slide 15).
- **P1 contro P1.5:** 8,69 contro 8,07 milioni al secondo, 115 contro 124 ns. Congelare il numero del nodo nel codice fa risparmiare circa 9 ns per pacchetto: poco, ma misurabile, perché fra i giri le cifre variano dell'1,2–1,3%.

La baseline arriva al suo limite appena: il generatore ne offre 15,13 milioni al secondo e il nodo li elabora tutti lavorando al 100%. rxonly invece non satura (il nodo riceve 19 milioni al secondo e lavora al 77%): il suo valore, circa 40 ns, è indicativo.

Verificato il 2 ottobre: scartare costa più che inoltrare, circa 50 ns: nella misura per classe la classe DROP costa 168 ns su P1 contro 114–119, 473 su P3 contro 423–432. Quando il pacchetto viene scartato la sua memoria si libera sul core del nodo; quando viene inoltrato, su quello d'uscita. A 512 e 1 514 byte ogni pipeline costa come a 64 entro il 2,5%: il nodo tocca solo le intestazioni.

Con xdp_gen due core reggono il doppio

4 - I RISULTATI SUL DUAL BOOT - XDP_GEN

Con xdp_gen due core reggono il doppio

Milioni di pacchetti al secondo elaborati, con xdp_gen

■ 2 core, misurati ■ 2 × un core



Una coda d'uscita per ogni core

Prima i due core del nodo scrivevano nella stessa coda d'uscita, svuotata da un solo core: la baseline elaborava 13,1 M/s e ne arrivavano 4,3. Ora ogni core ha la sua coda e il suo core d'uscita: arrivano tutti.

Il costo per core non cambia

Tempo di CPU per pacchetto di ogni core: 67, 119, 127, 297 e 436 ns, contro 66, 115, 124, 293 e 416 con un core solo (slide 11).

QUINDI

Con uno scrittore per coda anche in uscita, due core elaborano il **95-99% del doppio** di un core: il costo misurato su un core vale per ogni core.

xdp_gen, 2 ottobre, 64 byte, 3 giri · nodo su CPU 6 e 8, un thread per coda, uscita su CPU 3 e 5 · 2 × un core = il doppio della slide 11

12

La slide dà al nodo **due core** (CPU 6 e 8), ciascuno con la sua coda d'ingresso alimentata da un thread di xdp_gen. Le barre colorate sono i pacchetti al secondo elaborati con due core; le barre grigie sono il riferimento: **il doppio di quello che regge un core** nella slide 11 (per P2: $2 \times 3,41 = 6,82$ milioni al secondo). Le due misure sono della stessa sessione, il 2 ottobre.

Come si legge. P2 con due core elabora 6,74 milioni al secondo, il riferimento è 6,82: il 99%. Le altre: baseline 29,20 contro 30,26 (96%), P1 16,84 contro 17,38 (97%), P1.5 15,72 contro 16,14 (97%), P3 4,58 contro 4,82 (95%).

Il riquadro in basso a destra lo dice nell'unità del costo: il tempo di CPU per pacchetto di ogni core è 67, 119, 127, 297 e 436 ns, contro 66, 115, 124, 293 e 416 con un core solo. Per baseline, P1, P1.5 e P2 la differenza è al massimo 4 ns; P3 paga 20 ns in più, dentro la sua variazione fra una sessione e l'altra. Il secondo core non rallenta il primo perché i contatori delle pipeline sono **per-core**: ogni core incrementa la sua copia, senza istruzioni atomiche, e chi legge somma. Al nodo successivo arrivano tutti i pacchetti elaborati (persi dopo XDP al massimo lo 0,04%).

Perché serve una coda d'uscita per ogni core. Il riquadro in alto a destra racconta il problema che il banco aveva fino al 30 settembre. Il redirect verso il cavo d'uscita sceglie la coda dal numero del core che inoltra (numero modulo numero di code). Il banco dava a ogni porta d'uscita **una coda sola**: i due core del nodo scrivevano nello stesso anello, e un solo core del nodo successivo lo svuotava. Due danni insieme:

1. **contesa**: lucchetto e righe di cache dell'anello passano da un core all'altro a ogni pacchetto, e il costo lo paga il core del nodo nel redirect;
2. **un solo core d'uscita non basta**: ricevere, contare e restituire la pagina costa al nodo successivo circa 70 ns a pacchetto, cioè un tetto di circa 10 milioni al secondo. Oltre, l'anello trabocca, il redirect fallisce e il pacchetto si perde dopo il nodo, che paga anche lo scarto.

Il problema e la soluzione, un passo alla volta (baseline, misure del 1 ottobre da leggere come confronto fra loro). Una coda d'uscita e un core d'uscita: 13,15 milioni al secondo elaborati, 4,2 arrivati (49% perso dopo XDP), 152 ns per core. Tre code d'uscita (il core 6 scrive nella 0, l'8 nella 2) e un core d'uscita: 19,15 elaborati, ne arrivano circa 10, 103 ns. Tre code e due core d'uscita (CPU 3 e 5, uno per coda): arrivano tutti, e il 2 ottobre la baseline elabora 29,20 milioni al secondo a 67 ns per core.

Perché tre code e non due? Con due code i core 6 e 8 cadrebbero tutti e due nella coda 0 (6 e 8 sono entrambi pari): il banco sceglie il numero minimo di code per cui i core del nodo cadono in code diverse. Con un solo core del nodo resta una coda, quindi le misure a un core non cambiano.

Conclusione: il costo per pacchetto resta quello misurato su un core, e un nodo con N code regge circa N volte la capacità di un core. Una scheda vera distribuisce i pacchetti su più code e più core (RSS), con un solo scrittore per coda per costruzione: la scheda stessa.

Come si risponde alla domanda: due programmi in catena

5 · LA DOMANDA · COME SI MISURA

Due programmi sullo stesso aggancio, tre conteggi

Un'interfaccia porta un solo programma XDP: i due si mettono **in catena**, con un salto.



inviati – ricevuti

= persi **prima** del programma
respinti dalla coda d'ingresso

ricevuti – inoltrati

= persi **dopo** il programma
o decisioni del modello (scarta)

rxonly = il solo programma 1

Dice quanto regge la sola ricezione allo stesso ritmo. Il programma 1 costa fino a +5 ns: dentro il rumore.

Il salto è la tail call che le pipeline già usano · costo del programma 1 misurato con le statistiche del kernel

13

Per sapere dove si perdono i pacchetti servono tre contatori, messi in tre punti diversi del percorso:

1. **inviati**: li conta xdp_gen;
2. **ricevuti**: li conta un piccolo programma messo davanti alla pipeline, che fa solo `ing_count += 1` e poi passa il pacchetto alla pipeline;
3. **inoltrati**: li conta il nodo successivo.

Un'interfaccia può avere un solo programma XDP agganciato. Per averne due, il primo (il contatore) salta al secondo (la pipeline) con una **tail call**, lo stesso meccanismo che le pipeline usano già al loro interno (P3 lo usa fra uno strato e l'altro). Il salto non torna indietro: la pipeline prosegue come se fosse stata agganciata direttamente.

Con i tre conteggi la perdita si divide in due parti:

- **inviati – ricevuti** = persi prima del programma, cioè respinti dalla coda d'ingresso piena;
- **ricevuti – inoltrati** = persi dopo il programma, oppure pacchetti che il modello ha deciso di scartare (si separano con i contatori della pipeline, slide 14).

Esempio. In una finestra di misura: inviati 1 000 000, ricevuti 800 000, inoltrati 800 000. Allora 200 000 pacchetti si sono persi prima del programma (respinti dalla coda) e zero dopo.

Il contatore costa pochissimo, e lo si misura invece di supportarlo: il kernel tiene il tempo di ogni programma agganciato, e con e senza il contatore davanti la differenza è di pochi ns (+1 / +2 ns per P1 e baseline), dentro la variazione fra le ripetizioni. rxonly è il contatore da solo, senza pipeline dietro: dice quanto regge la sola ricezione allo stesso ritmo.

Come si leggono i tre conteggi

5 · LA DOMANDA · COME SI LEGGE

Dove mancano i pacchetti indica il collo di bottiglia

Un programma XDP non ha una coda sua: se è lento, si riempie la coda che sta **davanti** a lui.

SI OSSERVA

inviati > ricevuti

mancano pacchetti prima di XDP



VUOL DIRE

La coda d'ingresso è piena: il collo di bottiglia è **il programma** (o la sola ricezione: lo separa rxonly).

SI OSSERVA

ricevuti > inoltrati

al nodo successivo ne arrivano meno: si guarda che cosa aveva deciso il programma



SE AVEVA DECISO DI INOLTARE

Il pacchetto non è arrivato: il collo di bottiglia è **la trasmissione**.

SE AVEVA DECISO DI SCARTARE (O NON USATA)

Non è una perdita: è **una decisione** della rete neurale.

QUINDI

Tre conteggi più le decisioni del programma separano **tre casi** che altrimenti sembrerebbero uguali.

Inviati: generatore · ricevuti: programma 1 · inoltrati: nodo successivo · decisioni: contatori della pipeline (HIT, MISS, DROP)

14

Un programma XDP non ha una coda sua: elabora un pacchetto alla volta, dall'inizio alla fine, sul core che lo ha ricevuto. Se è lento, non accumula lavoro dentro di sé: si riempie la coda che sta davanti a lui. Per questo il punto in cui mancano i pacchetti dice chi è il collo di bottiglia. La slide distingue tre casi:

- **inviati > ricevuti**: mancano pacchetti prima di XDP, la coda d'ingresso era piena. Il collo di bottiglia è il programma, oppure la sola ricezione: lo si capisce confrontando con rxonly (slide 18).
- **ricevuti > inoltrati, e il programma aveva deciso di inoltrare**: il pacchetto è stato elaborato ma non è arrivato. Il collo di bottiglia è la trasmissione in uscita.
- **ricevuti > inoltrati, e il programma aveva deciso di scartare** (o la classe non è usata): non è una perdita, è la rete neurale che ha scelto così.

Le decisioni si leggono dai contatori che ogni pipeline tiene comunque: HIT (inoltrati secondo il modello), MISS (nessuna regola trovata) e DROP (scartati per decisione).

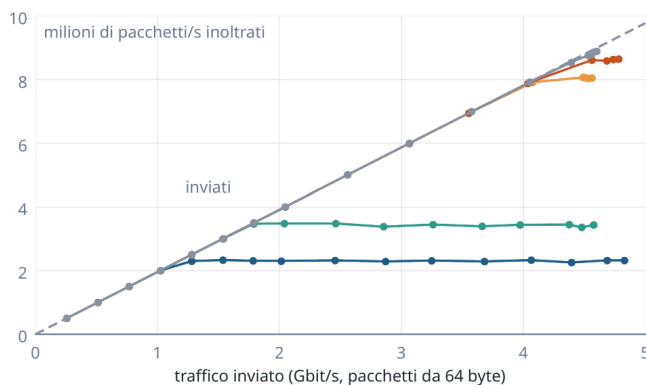
Esempio. Ricevuti 100, inoltrati 90. Se il programma ha deciso "scarta" per 10 pacchetti, va tutto bene: zero perdite. Se invece aveva deciso "inoltra" per tutti e 100, 10 si sono persi in uscita.

Tre conteggi più le decisioni del programma separano quindi tre casi che, guardando solo inviati e arrivati, sembrerebbero uguali.

Le curve: pacchetti inoltrati al crescere del traffico

5 - LA DOMANDA - PACCHETTI INOLTRATI

Ogni curva sale con il traffico e poi si ferma



Dove si ferma ogni pipeline

pipeline	pulita fino a (Gbit/s)	perde da (Gbit/s)	inoltrato max (M pacchetti/s)
baseline	$\geq 4,05$	—	8,89*
P1	4,04	4,56	8,64
P1.5	3,55	4,08	8,06
P2	1,54	1,79	3,48
P3	1,02	1,28	2,33

* Baseline: oltre 8,6 M/s il nodo è al 60%: il limite è del banco, non del programma.

QUINDI

Ogni curva si ferma alla **capacità della sua pipeline**: P1 a 8,6, P1.5 a 8,1, P2 a 3,5 e P3 a 2,3 milioni di pacchetti al secondo, le stesse cifre del test di saturazione (slide 11).

16 valori di traffico, 5 giri ciascuno, 64 byte, 480 finestre · xdp_gen, ritmo tenuto dal generatore · DUT a 3,5 GHz

15

Come si legge il grafico. In orizzontale c'è il traffico inviato (in Gbit/s, con pacchetti da 64 byte), in verticale i pacchetti che il nodo riesce a inoltrare (milioni al secondo). La linea tratteggiata è "tutto quello che si invia": finché una curva sta sulla diagonale, il nodo inoltra tutto. Quando la curva diventa piatta, il nodo è arrivato alla sua capacità e il resto si perde.

Da Gbit/s a pacchetti. Un pacchetto da 64 byte è 512 bit. Quindi $1 \text{ Gbit/s} = 1\,000\,000\,000 / 512 \approx 1,95$ milioni di pacchetti al secondo. P2 si ferma a 3,48 milioni di pacchetti al secondo, cioè $3,48 \times 0,512 \approx 1,78$ Gbit/s.

Tutte e quattro le pipeline si fermano alla capacità trovata nel test di saturazione (slide 11): P1 a 8,64 contro 8,69, P1.5 a 8,06 contro 8,07, P2 a 3,48 contro 3,41, P3 a 2,33 contro 2,41 milioni al secondo. Due test diversi, uno a traffico crescente e uno a traffico massimo, fatti in due momenti diversi della giornata, danno lo stesso numero entro il 3%: la capacità misurata è una proprietà del nodo e non del metodo. Al ginocchio il nodo lavora il 99,6–99,9% del tempo.

La baseline è un caso a parte. Da 9 milioni al secondo perde circa l'1%, ma il nodo lavora solo il 60% del tempo: ha tempo libero, quindi non è il programma a non farcela. È il banco: quando i pacchetti vengono inoltrati, xdp_gen non riesce a mandarne più di circa 9 milioni al secondo (senza inoltrato, con rxonly, arriva a 11,7), e i suoi lotti da 256 pacchetti a volte trovano la coda non ancora vuota. Per questo nella tabella la baseline ha l'asterisco: la sua capacità vera, 15,1 milioni al secondo, è quella della slide 11.

Le colonne della tabella. Il traffico si prova a 16 valori, da 0,5 a 12 milioni di pacchetti al secondo. "Pulita fino a" è l'ultimo valore provato senza perdite, "perde da" il primo con perdite: la soglia vera sta fra i due.

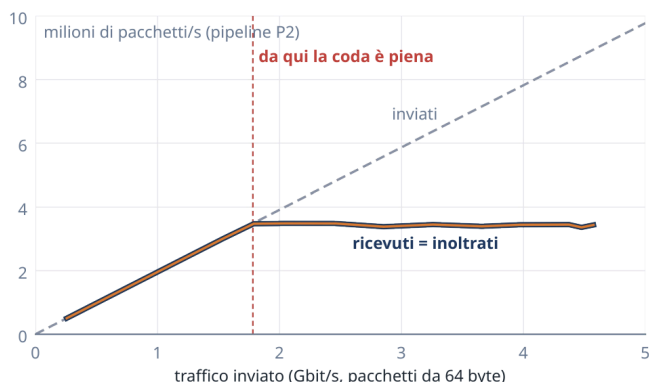
Esempio: P2 è pulita a 1,54 Gbit/s (3 milioni al secondo) e perde da 1,79 (3,5 milioni); la sua soglia vera è 1,78. "Inoltro max" è il massimo che la pipeline ha inoltrato.

Ogni punto è la mediana di 5 giri: $16 \text{ valori} \times 5 \text{ giri} \times 6 \text{ programmi} = 480$ finestre di misura, con il nodo a 3,5 GHz.

Ricevuti e inoltrati coincidono

5 · LA DOMANDA · RICEVUTI E INOLTRATI

Il programma rilancia tutto ciò che riceve



1

Sotto il limite, le curve coincidono

Tutto ciò che si invia viene ricevuto e inoltrato.

2

Sopra il limite, salgono solo gli inviati

Ricevuti e inoltrati si fermano insieme, a 3,5 milioni al secondo.

3

Ciò che manca non è mai entrato

Il programma non perde niente di quello che prende.

QUINDI

Ricevuti e inoltrati restano sovrapposti in tutte e cinque le pipeline: dopo XDP si perde al massimo lo 0,04% degli inviati (baseline 0,2%, sopra 8,6 M/s).

Esempio: P2, mediana di 5 giri, xdp_gen · linea scura = ricevuti da XDP, arancione = inoltrati; coincidono

16

Il grafico mostra una sola pipeline, P2, con tre curve: inviati (tratteggiata), ricevuti dal programma (linea scura) e inoltrati (arancione). Ricevuti e inoltrati sono una sopra l'altra per tutto il grafico: tutto quello che il programma riceve, lo rilancia.

Si leggono tre fasi:

1. **sotto il limite** le tre curve coincidono: tutto ciò che si invia viene ricevuto e inoltrato;
2. **sopra il limite** sale solo la curva degli inviati; ricevuti e inoltrati si fermano insieme a 3,5 milioni al secondo;
3. **ciò che manca non è mai entrato**: la distanza fra inviati e ricevuti è tutta perdita all'ingresso.

Esempio con i numeri veri. A 2,85 Gbit/s si inviano 5,57 milioni di pacchetti al secondo. P2 ne riceve 3,38 e ne inoltra 3,38. I 2,19 milioni mancanti si sono persi tutti prima del programma, alla coda d'ingresso.

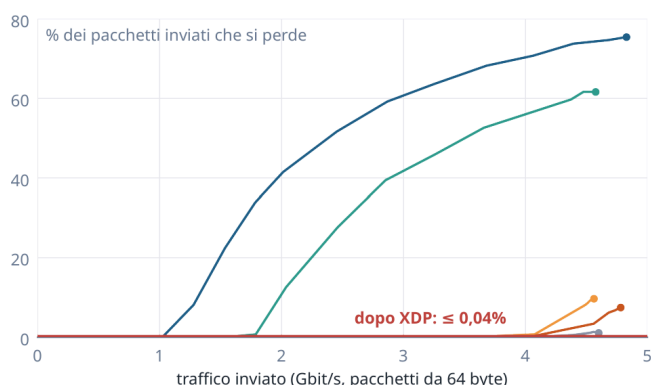
La linea rossa verticale segna il punto da cui la coda è piena: a sinistra le tre curve coincidono, a destra sale solo quella degli inviati. Lo stesso vale per tutte le pipeline, non solo per P2.

Sullo "0,04%". È rumore di misura, non una perdita vera. Il banco legge i contatori uno dopo l'altro, non tutti nello stesso istante, e in quel breve intervallo passano alcune centinaia di pacchetti. Su 400 misure il valore è negativo in 159 casi (una perdita vera non può essere negativa) e in media è quasi zero. Più corretto dire: dopo il programma la perdita è zero, entro il rumore di misura. Fa eccezione la baseline sopra 8,6 milioni al secondo (0,1-0,2%), per la ragione della slide 15.

La perdita è tutta prima del programma

5 · LA DOMANDA · LA PERDITA

La perdita è tutta prima del programma



Perdita al traffico più alto

prima del programma, % degli inviati

P3	76%
P2	61%
P1.5	9%
P1	7%
baseline	1%*
rxonly (solo ricezione)	0,3%

Dopo il programma: ≤ 0,04%

a ogni traffico (* baseline: banco, nodo al 60%; 0,2% dopo)

QUINDI

Se si perde all'ingresso il collo di bottiglia è il programma: qui la perdita è **tutta all'ingresso**, quindi il collo di bottiglia non è la trasmissione.

Perdita = (inviati - inoltrati) / inviati · mediana di 5 giri · si chiedono fino a 12 M pacchetti/s; a coda piena il generatore ne invia 8,7-9,4

17

Qui si guarda la percentuale di pacchetti persi, cioè $(\text{inviati} - \text{inoltrati}) / \text{inviati}$. Ogni curva resta a zero finché la pipeline regge, poi sale: più traffico si manda oltre la capacità, più grande è la parte che non entra. La curva sale tanto prima quanto più la pipeline è costosa.

Esempio di calcolo. All'ultimo punto a P3 arrivano 9,43 milioni di pacchetti al secondo e ne inoltra 2,32. La parte persa è $1 - 2,32 / 9,43 \approx 75\%$. P2: $1 - 3,36 / 8,74 \approx 62\%$.

Al traffico più alto: P3 76%, P2 61%, P1.5 9%, P1 7%, baseline 1% (il banco, slide 15), rxonly 0,3%. L'ordine è quello della capacità della slide 11. Si chiedono fino a 12 milioni al secondo, ma a coda piena ne partono 8,7-9,4: quando la coda respinge, ogni pacchetto respinto costa un po' anche al generatore, che rallenta.

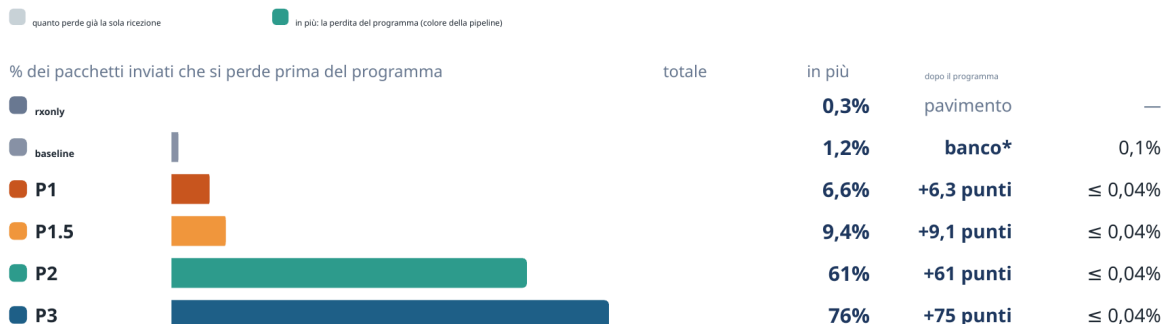
La linea rossa in basso è la perdita dopo il programma: resta a zero per tutte le pipeline e a ogni traffico (lo "0,04%" è rumore, vedi slide 16). Quindi il collo di bottiglia è il programma, non la trasmissione.

È il programma o la sola ricezione?

5 · LA DOMANDA · IL PROGRAMMA O LA RICEZIONE?

La sola ricezione non perde: la perdita è del programma

Fino a 11 milioni di pacchetti al secondo il solo contatore (rxonly) non perde niente; a 12 milioni perde lo 0,3%.



QUINDI

La perdita cresce con il costo del programma: 7% per P1, 9% per P1.5, 61% per P2, 76% per P3. *La baseline perde l'1% con il nodo al 60%: è il banco, non il programma.

Traffico più alto chiesto: 12 M pacchetti/s · xdp_gen · rxonly su un core regge 19 M/s: qui non arriva al suo limite

18

La perdita all'ingresso potrebbe venire dalla sola ricezione, che gira sullo stesso core del programma. Lo si controlla con rxonly, il contatore da solo: **fino a 11 milioni di pacchetti al secondo rxonly non perde niente**; a 12 milioni perde lo 0,3%, dove comincia a cedere il generatore stesso. Il pavimento, cioè la parte di perdita che non dipende dalla pipeline, è praticamente zero.

Ogni barra è quindi quasi tutta "in più": è perdita dovuta al lavoro del programma. 7% per P1, 9% per P1.5, 61% per P2, 76% per P3: cresce con il costo del programma. La baseline perde l'1%, ma con il nodo al 60%: non è il programma (slide 15). La colonna "dopo il programma" resta a zero per tutte le pipeline.

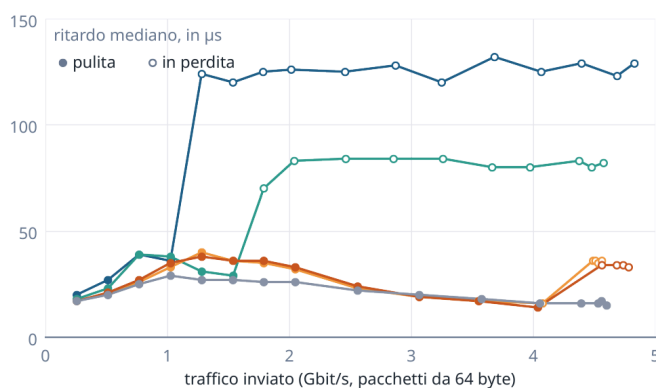
Perché il pavimento è zero. Su un core, senza la copia, la sola ricezione regge circa 19 milioni di pacchetti al secondo (slide 11), più del traffico più alto di questa scala. Le pipeline cominciano a perdere molto prima (P3 già a 2,5 milioni al secondo, P2 a 3,5, P1.5 a 8, P1 a 9): nella zona in cui le pipeline perdono, la ricezione da sola non è vicina al suo limite.

Con pktgen il pavimento non era zero. Con l'alternativa la ricezione costa di più (c'è la copia) e regge circa 4,6 milioni al secondo: allora anche rxonly perdeva il 21% al traffico più alto, e la perdita del programma si leggeva come differenza. Con xdp_gen la risposta è più netta.

Il ritardo

5 · LA DOMANDA · IL RITARDO

Il ritardo salta dove comincia la perdita



La coda piena spiega il ritardo

attesa prevista = 256 posti + capacità della pipeline

pipeline	prevista	misurata (p50)
baseline	—	—
P1	30 μs	34 μs
P1.5	32 μs	36 μs
P2	74 μs	83 μs
P3	110 μs	125 μs

Il resto (10-15%) è il percorso fino al nodo successivo; — = coda mai piena.

QUINDI

Oltre la sua capacità il nodo non riesce a svuotare la coda, che resta **piena**: il ritardo si ferma e cresce la perdita.

Ritardo dal timbro di xdp_gen al nodo successivo · p50 · — = la baseline non riempie la coda · a basso carico c'è l'attesa nel generatore

19

Il ritardo è il tempo dal momento in cui xdp_gen spedisce il pacchetto (lo scrive nel pacchetto stesso, come un timbro) a quando arriva al nodo successivo. Il grafico mostra la mediana, cioè il valore "tipico". I pallini pieni sono i punti senza perdita, quelli vuoti i punti in cui perde il programma.

- Finché il nodo regge, il ritardo è di qualche decina di microsecondi, e non cresce con il traffico.
- Quando comincia la perdita il ritardo salta di colpo: 34 μs per P1, 36 per P1.5, 83 per P2, 125 per P3. La coda è piena, e ogni pacchetto deve aspettare che passino i 255 davanti a lui.
- Poi resta piatto: la coda non può diventare più lunga di 256 posti. Il traffico in più non aumenta l'attesa, viene semplicemente respinto.

Esempio di calcolo. P2 elabora 3,48 milioni di pacchetti al secondo, cioè uno ogni 287 ns. Con la coda piena un pacchetto aspetta $256 \times 287 \text{ ns} \approx 74 \mu\text{s}$. La misura dà 83 μs : il resto (circa il 10%) è il viaggio fino al nodo successivo.

La tabella a destra fa lo stesso conto per tutte: P1 30 previsti contro 34 misurati, P1.5 32 contro 36, P2 74 contro 83, P3 110 contro 125. Previsto e misurato sono vicini, quindi il salto del ritardo è proprio la coda piena. La baseline non riempie mai la coda (il nodo ha tempo libero, slide 15): per lei la riga è vuota.

Il ritardo a basso carico. I 17-19 μs a sinistra contengono un'attesa dentro il generatore: xdp_gen spedisce a lotti, e il nodo viene svegliato a fine lotto, quindi un pacchetto può aspettare fino a una dozzina di μs prima di essere visto. Con lotti più piccoli (32 invece di 256) il ritardo a basso carico scende a 3-6 μs , ma il generatore regge il ritmo solo fino a 2,5 milioni al secondo. Il salto a coda piena non cambia.

In sintesi

IN CHIUSURA · SINTESI

In sintesi

1

Il dual boot rende le misure ripetibili

La variazione del costo scende da 15-160% a 2-4%. Il core del nodo non dorme, non cambia frequenza e nessuno lo interrompe.

2

Come una scheda vera

xdp_gen consegna frame senza copia: P1-P3 da 115 a 416 ns, e due core reggono il doppio. pktgen resta un'alternativa.

3

Si perde prima del programma

Il nodo satura alla sua capacità, il ritardo sale alla coda piena e la perdita è tutta all'ingresso: il collo di bottiglia è il programma, e due core reggono il doppio.

Ettore Cantile · ottobre 2026

20

- 1. Il dual boot rende le misure ripetibili.** Nella macchina virtuale lo stesso costo variava fino al 160% fra una volta e l'altra; ora del 2-4%, e si vedono anche differenze piccole come P1 contro P1.5 (9 ns) o scartare contro inoltrare (50 ns).
- 2. Un generatore come una scheda vera.** xdp_gen consegna i pacchetti senza copia: P1-P3 costano 115, 124, 293 e 416 ns per pacchetto, la baseline 66, e due core reggono il doppio di uno. pktgen resta un'alternativa: costa 225-250 ns in più per la copia, ma sopra la baseline dà la stessa risposta.
- 3. Si perde prima del programma.** Quando il traffico supera la capacità, la coda si riempie, il ritardo salta alla coda piena e i pacchetti vengono respinti all'ingresso; dopo il programma non se ne perde nessuno, e la sola ricezione non perde affatto. Il collo di bottiglia è il programma.

Cose da dire con cautela. Slide 8: la colonna della macchina virtuale è misurata con pktgen a 512 byte, quella del dual boot con xdp_gen a 64 byte: si confronta solo la variazione. Slide 11: la baseline satura appena, rxonly no. Slide 15-19: la baseline perde da 9 milioni al secondo con il nodo al 60%: è un limite del banco, non del programma, e la sua capacità è quella della slide 11. Slide 16-18: " $\leq 0,04\%$ " è rumore di misura, meglio dire "zero". Slide 19: il ritardo a basso carico contiene l'attesa nel generatore. P3 varia fino al 10% fra una sessione e l'altra (416-460 ns).

Risolve dal 30 settembre al 2 ottobre. Il "tetto del generatore" con xdp_gen non era del generatore ma della coda d'ingresso condivisa: un thread per coda. Il nodo "occupato all'87%" con xdp_gen era una pausa di circa 12 ms del generatore a ogni chiamata al kernel: ora una chiamata per misura, nodo al 100%. Con due core i core del nodo scrivevano in una coda d'uscita sola, svuotata da un solo core: ora una coda e un core d'uscita per ogni core del nodo. xdp_gen tiene un ritmo fissato e timbra i pacchetti, quindi anche le curve al crescere del traffico sono fatte con lui.